Class 7: Machine Learning 1

Sofia Jaravata (A19160915)

Table of contents

Background

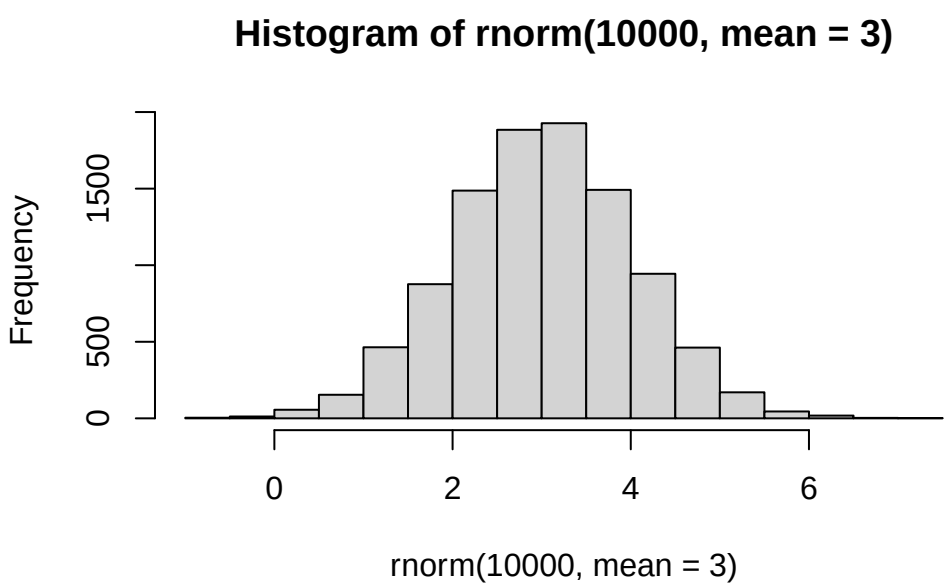
Today we will explore some core machine learning methods that are very popular in bioinformatics. These include **clustering** and **dimensionality reduction**.

K-means clustering

The main function in "base" R for K-means clustering is called `k-means()`

Before we go too deep let's make up some "simple" data that we can cluster and know if we are getting a good answer or not. To do this we can us the `rnorm()` function:

```
hist(rnorm(10000, mean =3))
```




```r
x <- c( rnorm(30, -3), rnorm(30, +3) )
x
```

```
 [1] -2.968037 -3.554327 -4.813653 -2.776197 -2.776357 -3.304721 -2.870752
 [8] -3.389540 -3.602803 -2.091119 -5.336096 -3.520502 -2.333758 -1.933002
[15] -2.172284 -2.483997 -1.726865 -1.269827 -3.540773 -2.611044 -3.040081
[22] -2.224436 -2.049749 -2.664620 -3.195888 -3.180052 -3.285722 -2.697093
[29] -2.525284 -2.329600  3.517458  2.858181  4.775000  2.950010  3.762120
[36]  2.916808  3.034508  3.264178  2.294709  2.649086  1.887488  3.917853
[43]  3.103798  4.371709  1.268144  2.227563  2.452515  5.388723  2.272942
[50]  2.858772  2.641549  2.158054  1.990416  3.247048  2.875768  3.133036
[57]  1.893332  3.579165  1.455721  2.274969
```

```r
rev(x)
```

```
 [1]  2.274969  1.455721  3.579165  1.893332  3.133036  2.875768  3.247048
 [8]  1.990416  2.158054  2.641549  2.858772  2.272942  5.388723  2.452515
[15]  2.227563  1.268144  4.371709  3.103798  3.917853  1.887488  2.649086
[22]  2.294709  3.264178  3.034508  2.916808  3.762120  2.950010  4.775000
[29]  2.858181  3.517458 -2.329600 -2.525284 -2.697093 -3.285722 -3.180052
[36] -3.195888 -2.664620 -2.049749 -2.224436 -3.040081 -2.611044 -3.540773
[43] -1.269827 -1.726865 -2.483997 -2.172284 -1.933002 -2.333758 -3.520502
```

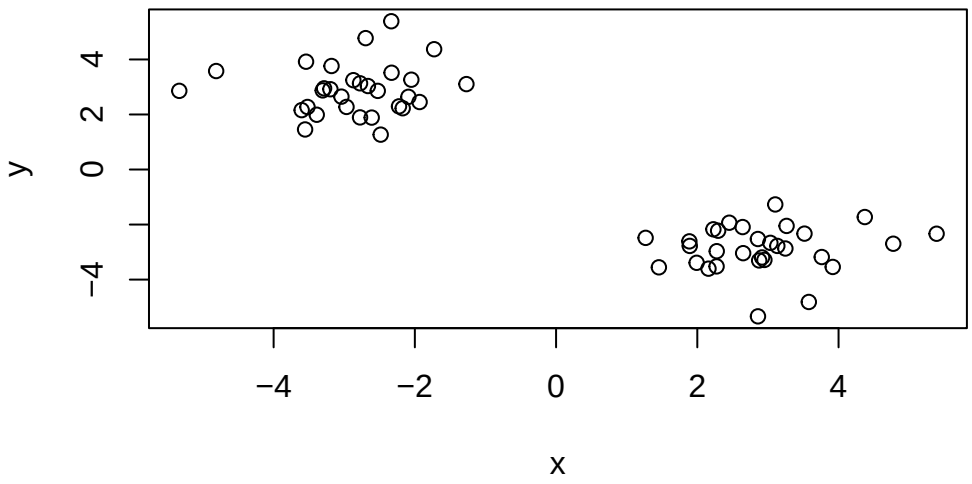
```
[50] -5.336096 -2.091119 -3.602803 -3.389540 -2.870752 -3.304721 -2.776357
[57] -2.776197 -4.813653 -3.554327 -2.968037
```

```
z <- cbind(x=x, y=rev(x))
z
```

```
              x         y
 [1,] -2.968037  2.274969
 [2,] -3.554327  1.455721
 [3,] -4.813653  3.579165
 [4,] -2.776197  1.893332
 [5,] -2.776357  3.133036
 [6,] -3.304721  2.875768
 [7,] -2.870752  3.247048
 [8,] -3.389540  1.990416
 [9,] -3.602803  2.158054
[10,] -2.091119  2.641549
[11,] -5.336096  2.858772
[12,] -3.520502  2.272942
[13,] -2.333758  5.388723
[14,] -1.933002  2.452515
[15,] -2.172284  2.227563
[16,] -2.483997  1.268144
[17,] -1.726865  4.371709
[18,] -1.269827  3.103798
[19,] -3.540773  3.917853
[20,] -2.611044  1.887488
[21,] -3.040081  2.649086
[22,] -2.224436  2.294709
[23,] -2.049749  3.264178
[24,] -2.664620  3.034508
[25,] -3.195888  2.916808
[26,] -3.180052  3.762120
[27,] -3.285722  2.950010
[28,] -2.697093  4.775000
[29,] -2.525284  2.858181
[30,] -2.329600  3.517458
[31,]  3.517458 -2.329600
[32,]  2.858181 -2.525284
[33,]  4.775000 -2.697093
[34,]  2.950010 -3.285722
[35,]  3.762120 -3.180052
```

```
[36,]  2.916808 -3.195888
[37,]  3.034508 -2.664620
[38,]  3.264178 -2.049749
[39,]  2.294709 -2.224436
[40,]  2.649086 -3.040081
[41,]  1.887488 -2.611044
[42,]  3.917853 -3.540773
[43,]  3.103798 -1.269827
[44,]  4.371709 -1.726865
[45,]  1.268144 -2.483997
[46,]  2.227563 -2.172284
[47,]  2.452515 -1.933002
[48,]  5.388723 -2.333758
[49,]  2.272942 -3.520502
[50,]  2.858772 -5.336096
[51,]  2.641549 -2.091119
[52,]  2.158054 -3.602803
[53,]  1.990416 -3.389540
[54,]  3.247048 -2.870752
[55,]  2.875768 -3.304721
[56,]  3.133036 -2.776357
[57,]  1.893332 -2.776197
[58,]  3.579165 -4.813653
[59,]  1.455721 -3.554327
[60,]  2.274969 -2.968037
```

```
plot(z)
```



Now we can run `kmeans()` on this input `z` and see what the results look like.

```
km <- kmeans(z, centers = 2)
km
```

```
K-means clustering with 2 clusters of sizes 30, 30

Cluster means:
          x         y
1 -2.875606  2.900687
2  2.900687 -2.875606

Clustering vector:
 [1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2
[39] 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2

Within cluster sum of squares by cluster:
[1] 45.51048 45.51048
 (between_SS / total_SS =  91.7 %)

Available components:

[1] "cluster"      "centers"      "totss"        "withinss"     "tot.withinss"
```

```
[6] "betweenss"    "size"         "iter"         "ifault"
```

```
attributes (km)
```

```
$names
[1] "cluster"      "centers"      "totss"        "withinss"     "tot.withinss"
[6] "betweenss"    "size"         "iter"         "ifault"

$class
[1] "kmeans"
```

How many points are in each cluster?

```
km$size
```

```
[1] 30 30
```

What "component of your result object" details cluster assignment/membership?

```
km$cluster
```

```
 [1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2
[39] 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
```

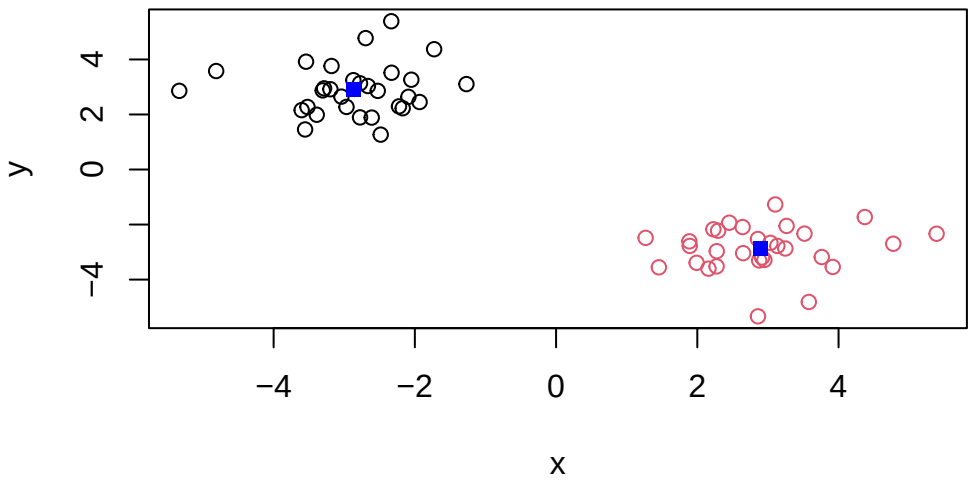
What "component of your result object" details cluster center?

```
km$centers
```

```
          x         y
1 -2.875606  2.900687
2  2.900687 -2.875606
```

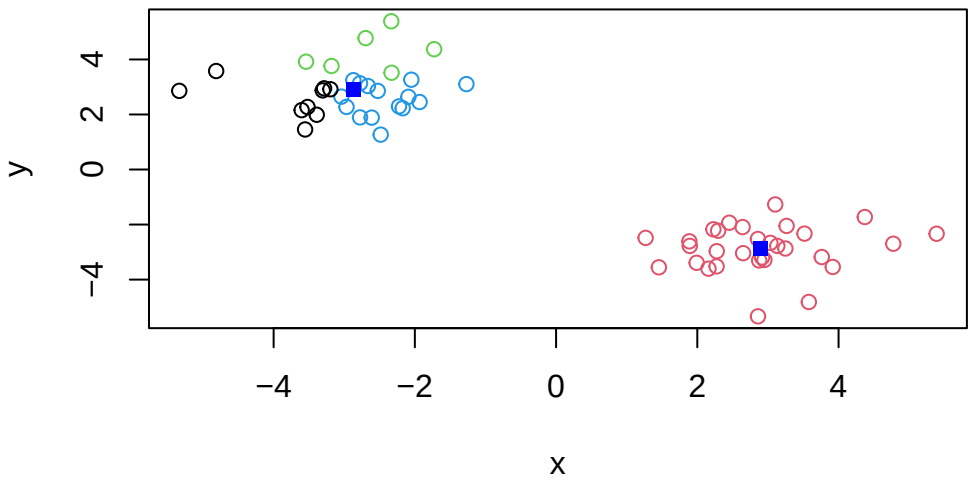
Q: Plot `z` colored by the kmeans cluster assignment and add cluster centers as blue points

```
plot(z, col = km$cluster)
points(km$centers, col = "blue", pch =15)
```



Q. Run a K-means clustering and plot the results asking for 4 clusters (k=4)?

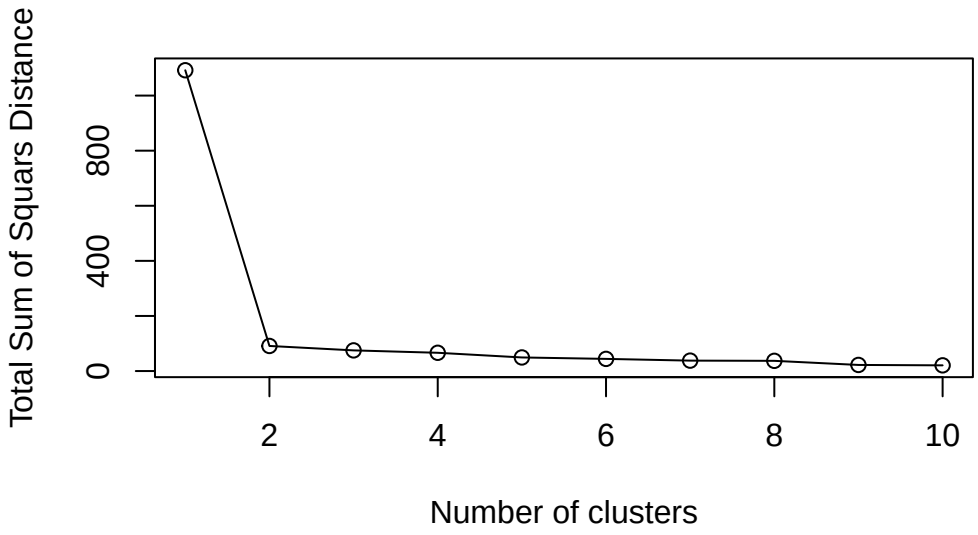
```
km4 <- (kmeans (z, centers = 4))
plot(z, col = km4$cluster)
points(km$centers, col = "blue", pch =15)
```



Note You need to tell K-means the number of clusters(i.e. set `centers=2`) !!

One approach is to try different values for `centers` and then pick the best…

```
ans <- NULL
for(i in 1:10){
km <-kmeans(z, centers = i)
ans <- c(ans, km$tot.withinss)

}

plot(ans, typ = "o",
     xlab = "Number of clusters",
     ylab = "Total Sum of Squars Distance")
```



Hierarchical Clustering

The main function in "base" R for Hierarchical Clustering is called `hclust()`

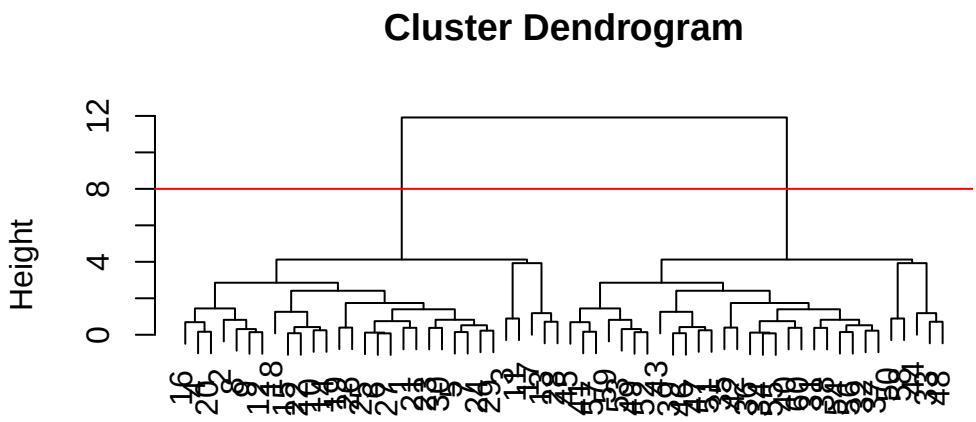
This function does not take your "Raw" data for clustering. You must first build a "distance" matrix from your data and pass this as input to `hclust()`

```
d <- dist(z)
hc <- hclust(d)
hc
```

```
Call:
hclust(d = d)

Cluster method   : complete
Distance         : euclidean
Number of objects: 60
```

There is a bespoke `plot()` method for `hclust()` result objects.

```
plot(hc)
abline(h=8, col = "red")
```



Once we have our `hclust` object (our "tree" of "cluster dendrogram"), we can *"cut"* the tree to reveal the clustering pattern with `cutree`.

```
cutree(hc, h=8)
```

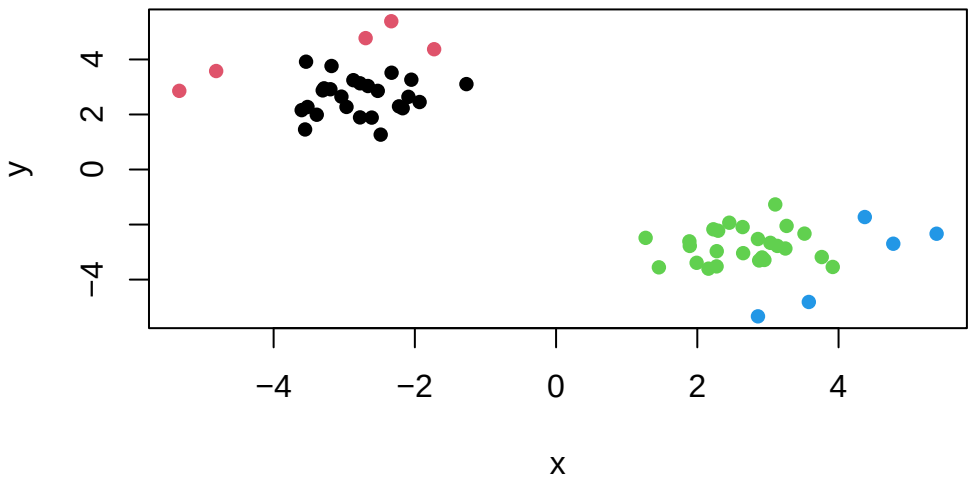
```
 [1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2
[39] 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
```

```
cutree(hc, k=4)
```

```
 [1] 1 1 2 1 1 1 1 1 1 1 2 1 2 1 1 1 2 1 1 1 1 1 1 1 1 1 2 1 1 3 3 4 3 3 3 3 3
[39] 3 3 3 3 3 4 3 3 3 4 3 4 3 3 3 3 3 3 3 4 3 3
```

Q. Make a plot of `z` with your hclust results (e. colored by cluster membership)

```
grps <- cutree(hc, k=4)
plot(z, col = grps, pch = 16)
```




```
#points(hc$cluster, col = "blue", pch =15)
```

Principal Component Analysis (PCA)

PCA is a dimensionality reduction method that is popular for revealing patterns in complex datasets.

Analysis of UK food data

Let's look at some data on the eating habits of folks from the UK to see if there are patterns and trends that have some regions being distinct from others.

Data Import

The data is made available in CSV format so we can use the `read.csv()` function for import to R.

```
url <- "https://tinyurl.com/UK-foods"
x <- read.csv(url)
x
```

```
                    X England Wales Scotland N.Ireland
1              Cheese     105   103      103        66
2        Carcass_meat     245   227      242       267
3          Other_meat     685   803      750       586
4                Fish     147   160      122        93
5       Fats_and_oils     193   235      184       209
6             Sugars     156   175      147       139
7      Fresh_potatoes     720   874      566      1033
8          Fresh_Veg     253   265      171       143
9          Other_Veg     488   570      418       355
10 Processed_potatoes     198   203      220       187
11     Processed_Veg     360   365      337       334
12       Fresh_fruit    1102  1137      957       674
13           Cereals    1472  1582     1462      1494
14         Beverages      57    73       53        47
15       Soft_drinks    1374  1256     1572      1506
16   Alcoholic_drinks     375   475      458       135
17     Confectionery      54    64       62        41
```

> Q1. How many rows and columns are in your new data frame named x? What R functions could you use to answer this questions?

```
ncol(x)
```

```
[1] 5
```

```
nrow(x)
```

```
[1] 17
```

```
dim(x)
```

```
[1] 17  5
```

There are 17 rows and 4 columns. We could use ncol() to view the number of both rows and columns, or nrow() and dim() to answer this question.

Checking your data

```
head(x)
```

```
              X England Wales Scotland N.Ireland
1        Cheese     105   103      103        66
2  Carcass_meat     245   227      242       267
3    Other_meat     685   803      750       586
4          Fish     147   160      122        93
5 Fats_and_oils     193   235      184       209
6        Sugars     156   175      147       139
```

```
rownames(x) <- x[,1]
x <- x[,-1]
head(x)
```

```
              England Wales Scotland N.Ireland
Cheese            105   103      103        66
Carcass_meat      245   227      242       267
Other_meat        685   803      750       586
Fish              147   160      122        93
Fats_and_oils     193   235      184       209
Sugars            156   175      147       139
```

```
dim(x)
```

```
[1] 17  4
```

```
x <- read.csv(url, row.names=1)
head(x)
```

```
              England Wales Scotland N.Ireland
Cheese            105   103      103        66
Carcass_meat      245   227      242       267
Other_meat        685   803      750       586
Fish              147   160      122        93
Fats_and_oils     193   235      184       209
Sugars            156   175      147       139
```

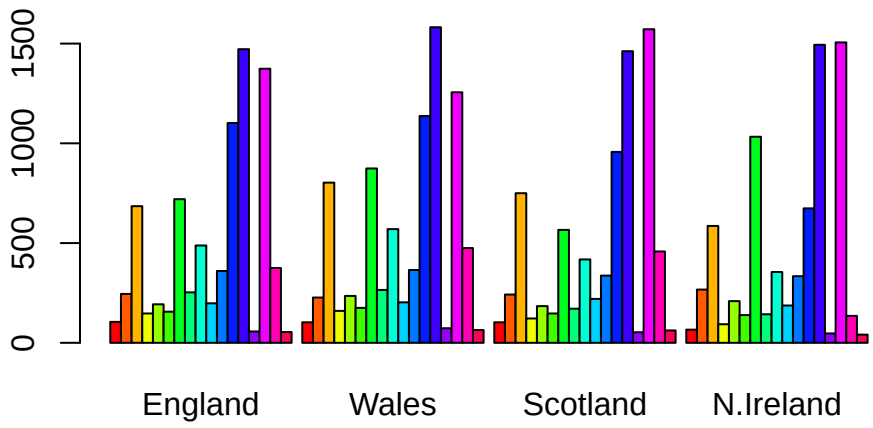
> Q2. Which approach to solving the 'row-names problem' mentioned above do you prefer and why? Is one approach more robust than another under certain circumstances?**

I prefer the second approach of read.csv(url, row.names=1) because it faster and more robust. Every time you run the first code, a column is lost, which does not happen in the second approach.

Spotting major differences and trends

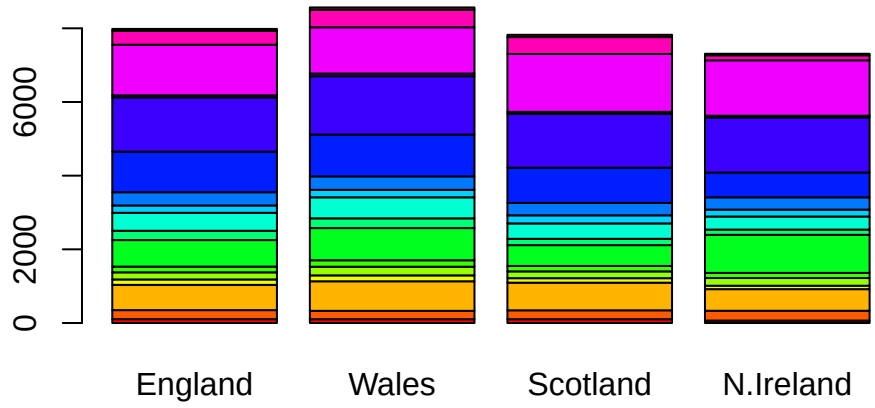
Make some plots to help make sense of obvious trends...

```
#Using base R
barplot(as.matrix(x), beside=T, col=rainbow(nrow(x)))
```



> Q3. Changing what optional argument in the above barplot() function results in the following plot?

```
barplot(as.matrix(x), beside=FALSE, col=rainbow(nrow(x)))
```



Changing "beside = T" to "beside = FALSE" would result in the following plot.

Tidy Data

Side-note: Using ggplot and the need for "tidy" data (Optional)

To convert this to "long" format we want one row per measurement - maximizes rows (17x4=68), minimizes columns (with a singe Consumption measurement value per Country). We will do tidying with the pivot_longer() function from the tidyr package:

```
library(tidyr)

# Convert data to long format for ggplot with `pivot_longer()`
x_long <- x |>
          tibble::rownames_to_column("Food") |>
          pivot_longer(cols = -Food,
                       names_to = "Country",
                       values_to = "Consumption")

dim(x_long)
```

```
[1] 68  3
```

```
head(x_long)
```

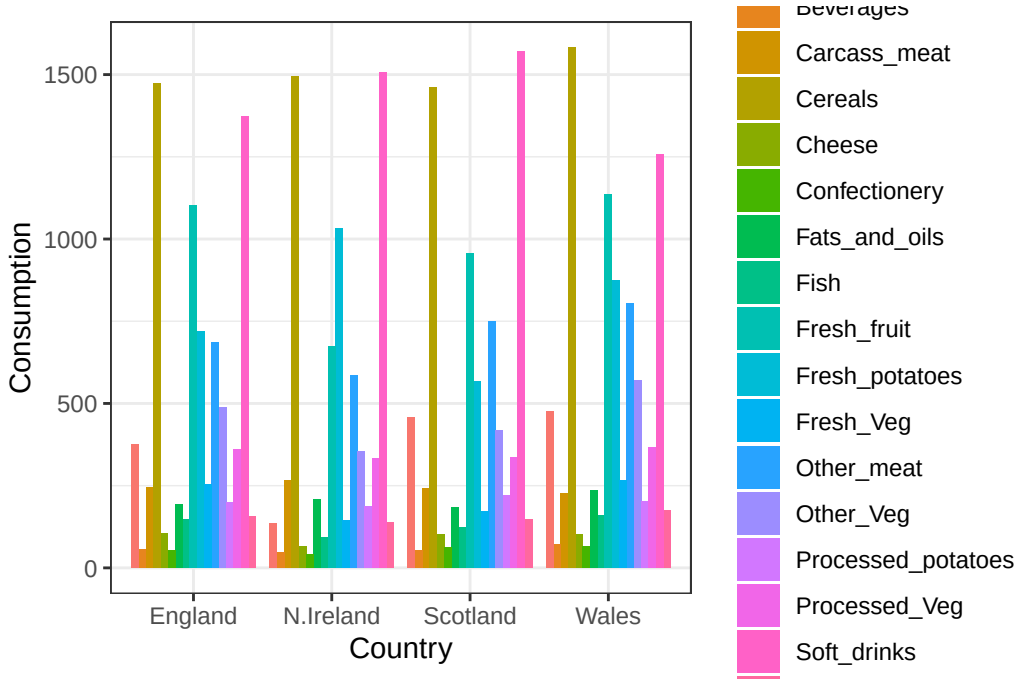
```
# A tibble: 6 x 3
  Food            Country   Consumption
  <chr>           <chr>           <int>
1 "Cheese"        England           105
2 "Cheese"        Wales             103
3 "Cheese"        Scotland          103
4 "Cheese"        N.Ireland          66
5 "Carcass_meat " England           245
6 "Carcass_meat " Wales             227
```

Grouped Bar Plot

```
# Create grouped bar plot
library(ggplot2)
```

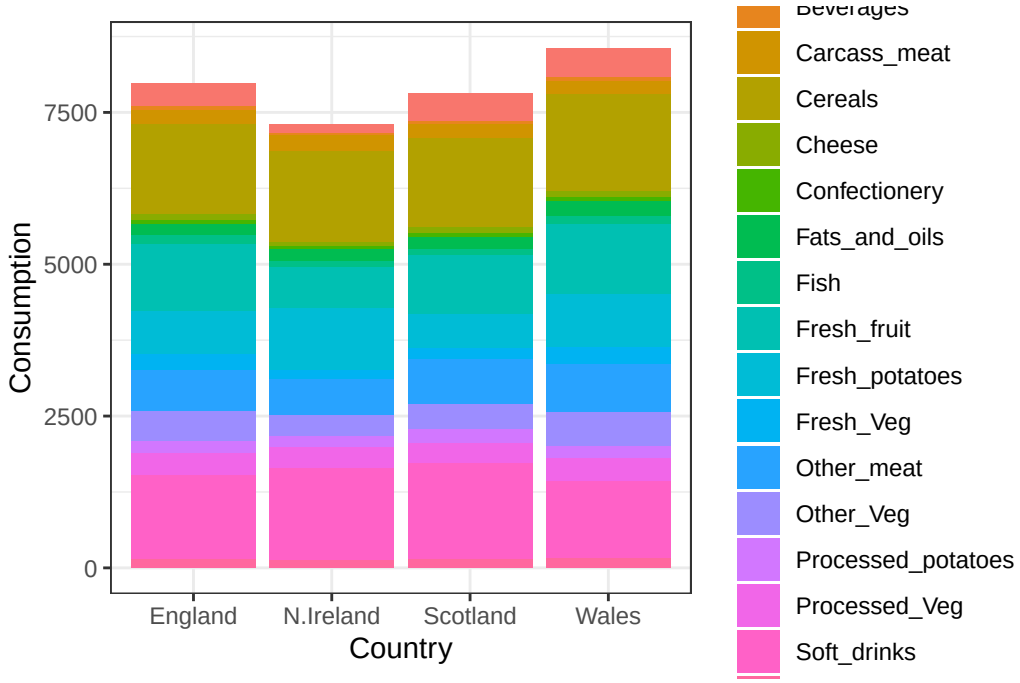
```
Warning: package 'ggplot2' was built under R version 4.5.2
```

```
ggplot(x_long) +
  aes(x = Country, y = Consumption, fill = Food) +
  geom_col(position = "dodge") +
  theme_bw()
```



> Q4. Changing what optional argument in the above ggplot() code results in a stacked barplot figure?

```
# Create grouped bar plot
library(ggplot2)
ggplot(x_long) +
  aes(x = Country, y = Consumption, fill = Food) +
  geom_col(position = "stack") +
  theme_bw()
```

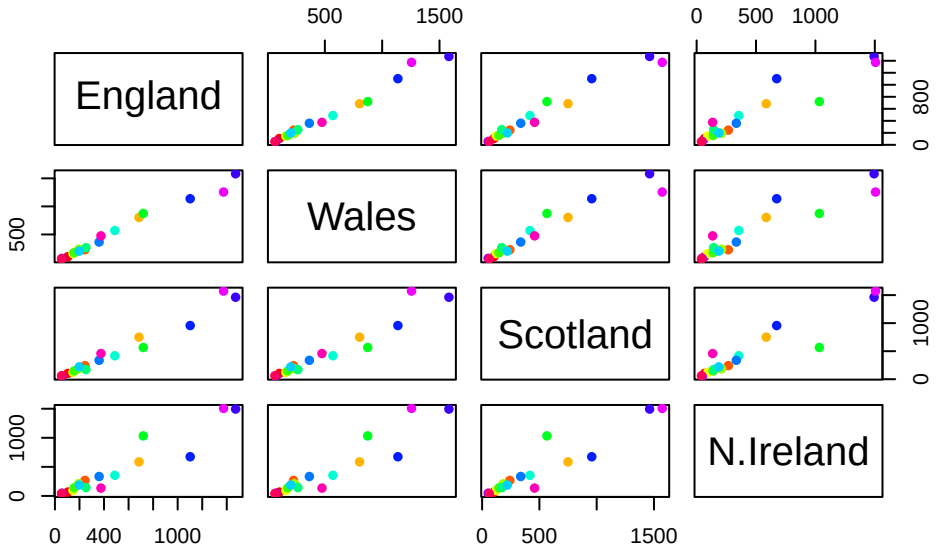


Changing the position in the geom_col() function from "dodge" to "stack" will result in the stacked barplot figure.

Pairs plots and heatmaps

> Q5. We can use the pairs() function to generate all pairwise plots for our countries. Can you make sense of the following code and resulting figure? What does it mean if a given point lies on the diagonal for a given plot?

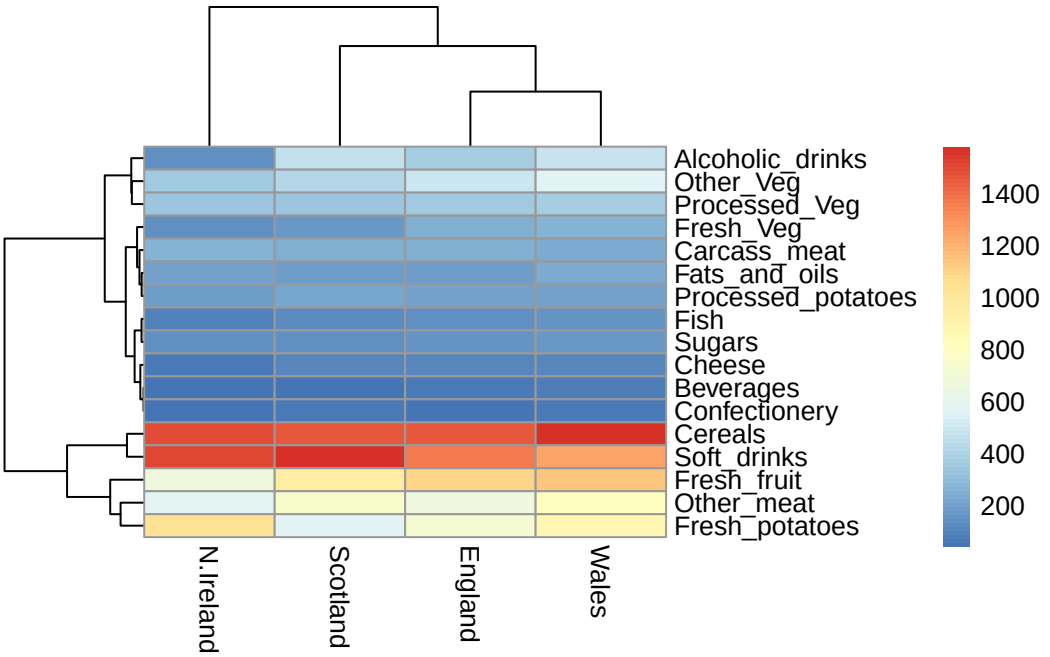
```
pairs(x, col=rainbow(nrow(x)), pch=16)
```



The following code results in matrices of different scatterplots with different x and y axes combinations of the countries: England, Wales, Scotland, and N. Ireland. For instance, in the first row, England is the y-axis for the 3 plots to the right, with Wales as the x-axis for the first plot (1st row, 2nd column), Scotland as the x-axis for the second plot (1st row, 3rd column), and N. Ireland as the x-axis for the third plot (1st row, 4rd column). A given point on the diagonal for a given plot indicates the equak amount of food being consumed by people from both countries.

heatmap

```
library(pheatmap)

pheatmap( as.matrix(x) )
```



> Q6. Based on the pairs and heatmap figures, which countries cluster together and what does this suggest about their food consumption patterns? Can you easily tell what the main differences between N. Ireland and the other countries of the UK in terms of this data-set?

Based on the pairs and heatmap figures, countries England and Walse cluster together. This suggests that their food consumption patterns are similar based on the similar colors in the heatmaps and consistent diagonals in the scatterplots. One difference between N. Ireland and the other countries of the UK in terms of this data-set is that N. Ireland has less data points on the right of the diagonal in the scatterplots when paired with other countries, and in the heatmap, N. Ireland consumes less of alcoholic drinks, fish, vegetables, fruit, and cheese but more fresh potatoes than the other countries.

Key-point: Even relatively small datasets can prove challenging to interpret.

PCA to the rescue

The main function in "base" R for PCA is called `prcomp()`. This function wants the "observations" to be rows and the "variables" to be columns. So here we need to take the transpose of our x input object.

```
t(x)
```

```
           Cheese Carcass_meat  Other_meat  Fish Fats_and_oils  Sugars
England       105           245         685  147            193    156
Wales         103           227         803  160            235    175
Scotland      103           242         750  122            184    147
N.Ireland      66           267         586   93            209    139
           Fresh_potatoes  Fresh_Veg  Other_Veg  Processed_potatoes
England               720        253        488                 198
Wales                 874        265        570                 203
Scotland              566        171        418                 220
N.Ireland            1033        143        355                 187
           Processed_Veg  Fresh_fruit  Cereals  Beverages Soft_drinks
England              360         1102     1472         57         1374
Wales                365         1137     1582         73         1256
Scotland             337          957     1462         53         1572
N.Ireland            334          674     1494         47         1506
           Alcoholic_drinks  Confectionery
England                 375             54
Wales                   475             64
Scotland                458             62
N.Ireland               135             41
```

The main function in "base R" for PCA is called `prompt()`. This function wants the observations to be rows and the variables to be columns.

```
pca <- prcomp(t(x))
summary(pca)
```

```
Importance of components:
                            PC1      PC2      PC3       PC4
Standard deviation     324.1502 212.7478 73.87622 2.921e-14
Proportion of Variance   0.6744   0.2905  0.03503 0.000e+00
Cumulative Proportion    0.6744   0.9650  1.00000 1.000e+00
```

The returned `pca` object has components that we can use to make our main result figures:

```
attributes(pca)
```

```
$names
[1] "sdev"     "rotation" "center"   "scale"    "x"

$class
[1] "prcomp"
```

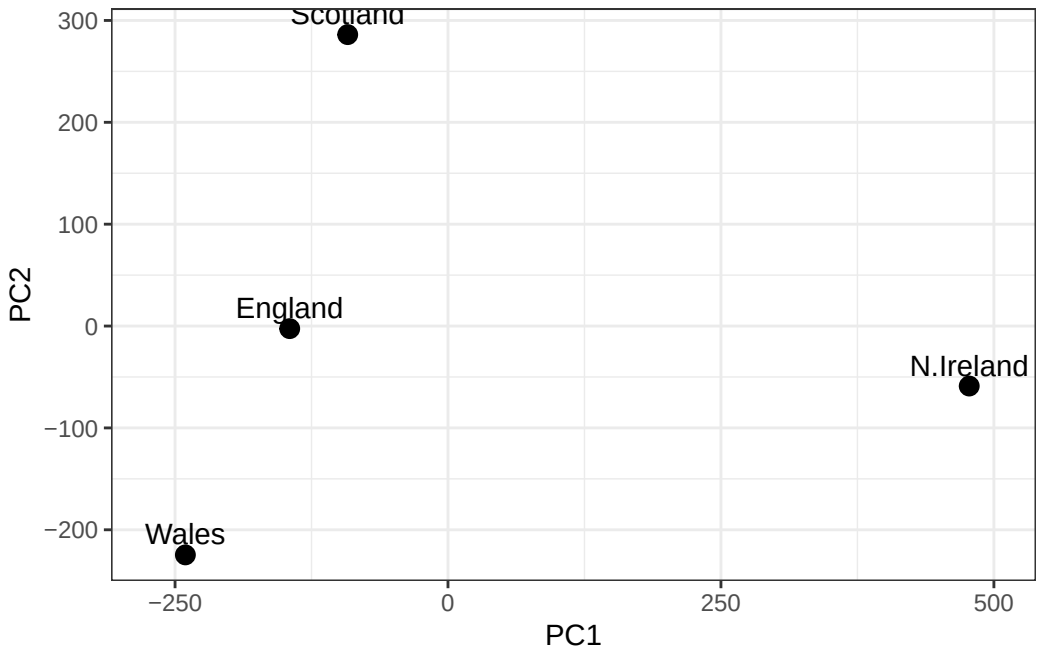
The main result figure from this analysis is called a **"PC score plot"** or "ordination plot" or "PC plot" or "PC1 vs PC2" plot.

```
pca$x
```

```
                PC1         PC2        PC3           PC4
England   -144.99315   -2.532999 105.768945 -9.152022e-15
Wales     -240.52915 -224.646925 -56.475555  5.560040e-13
Scotland   -91.86934  286.081786 -44.415495 -6.638419e-13
N.Ireland  477.39164  -58.901862  -4.877895  1.329771e-13
```

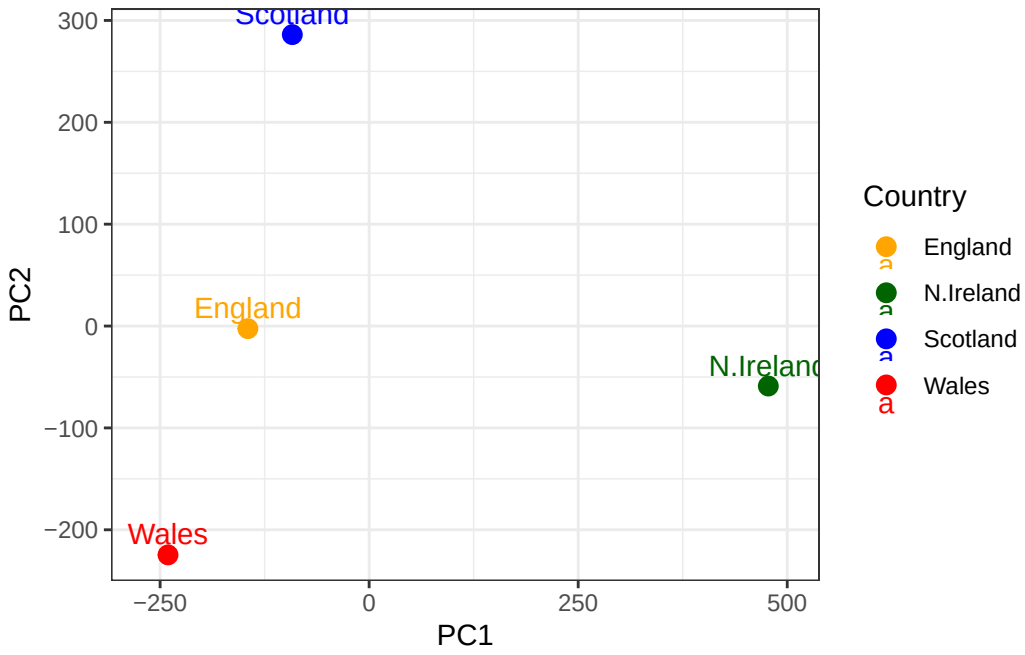
>Q7. Complete the code below to generate a plot of PC1 vs PC2. The second line adds text labels over the data points.

```
# Create a data frame for plotting
df <- as.data.frame(pca$x)
df$Country <- rownames(df)

# Plot PC1 vs PC2 with ggplot
ggplot(pca$x) +
  aes(x = PC1, y = PC2, label = rownames(pca$x)) +
  geom_point(size = 3) +
  geom_text(vjust = -0.5) +
  xlim(-270, 500) +
  xlab("PC1") +
  ylab("PC2") +
  theme_bw()
```



> **Q8. Customize your plot so that the colors of the country names match the colors in our UK and Ireland map and table at start of this document.**

```
df <- as.data.frame(pca$x)
df$Country <- rownames(df)

# Plot PC1 vs PC2 with ggplot
ggplot(df, aes(x = PC1, y = PC2, label = Country, color = Country)) +
  geom_point(size = 3) +
  geom_text(vjust = -0.5) +
  scale_color_manual(values = c(
    "England"  = "orange",
    "Scotland" = "blue",
    "Wales"    = "red",
    "N.Ireland"  = "darkgreen"
  )) +
  xlim(-270, 500) +
  xlab("PC1") +
  ylab("PC2") +
  theme_bw()
```



Below we can use the square of pca$sdev , which stands for "standard deviation", to calculate how much variation in the original data each PC accounts for.

```
v <- round( pca$sdev^2/sum(pca$sdev^2) * 100 )
v
```

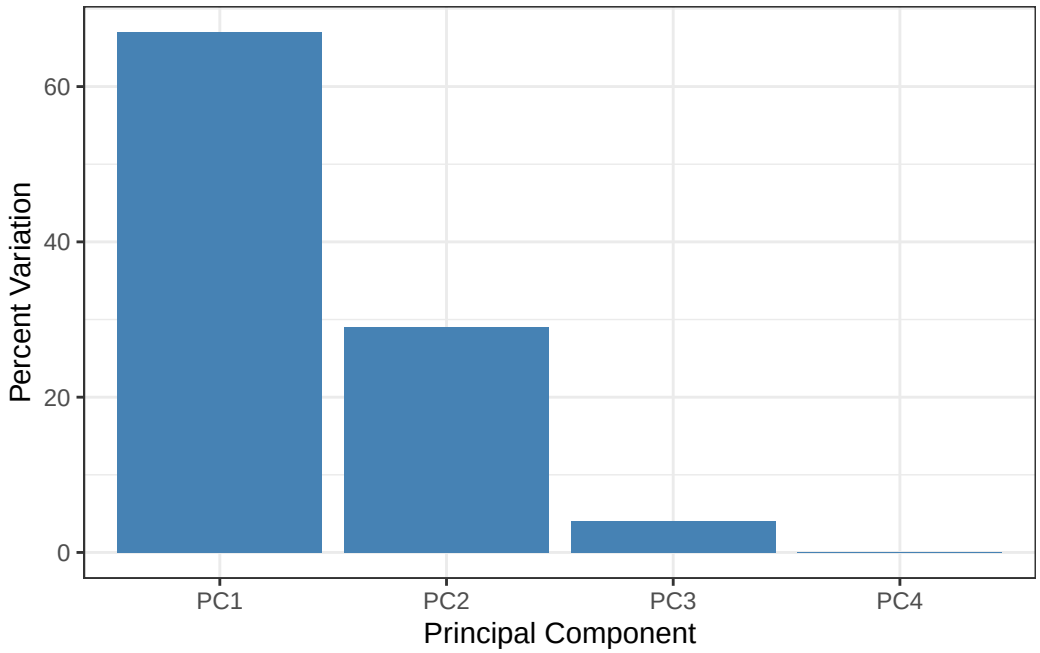
```
[1] 67 29  4  0
```

```
## or the second row here...
z <- summary(pca)
z$importance
```

```
                             PC1       PC2      PC3           PC4
Standard deviation     324.15019 212.74780 73.87622 2.921348e-14
Proportion of Variance   0.67444   0.29052  0.03503 0.000000e+00
Cumulative Proportion    0.67444   0.96497  1.00000 1.000000e+00
```

Scree plot with ggplot

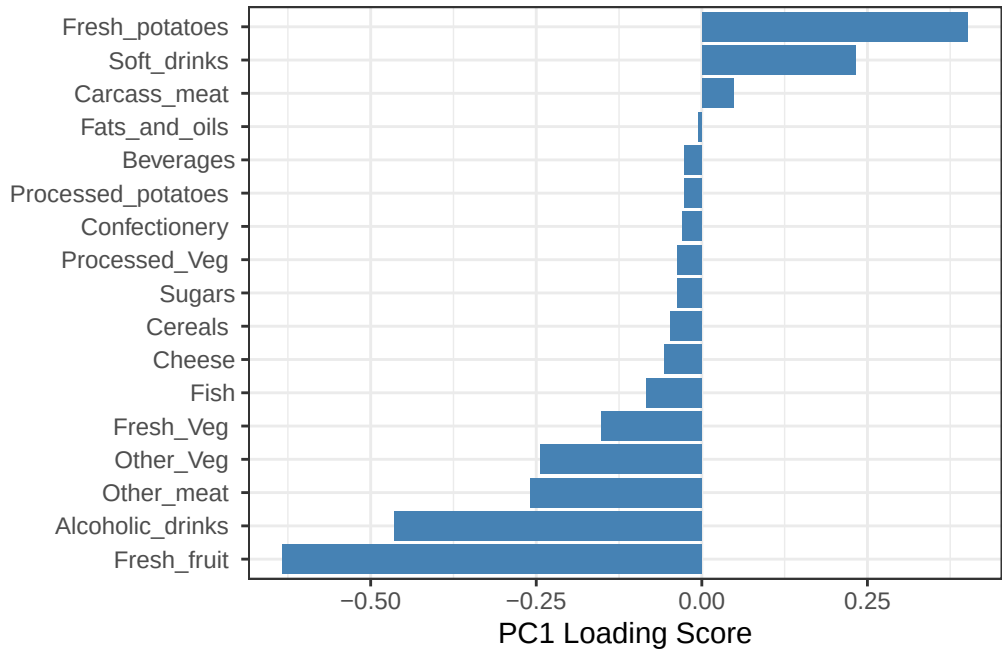
```r
# Create scree plot with ggplot
variance_df <- data.frame(
  PC = factor(paste0("PC", 1:length(v)), levels = paste0("PC", 1:length(v))),
  Variance = v
)

ggplot(variance_df) +
  aes(x = PC, y = Variance) +
  geom_col(fill = "steelblue") +
  xlab("Principal Component") +
  ylab("Percent Variation") +
  theme_bw() +
  theme(axis.text.x = element_text(angle = 0))
```



Digging deeper (variable loadings)

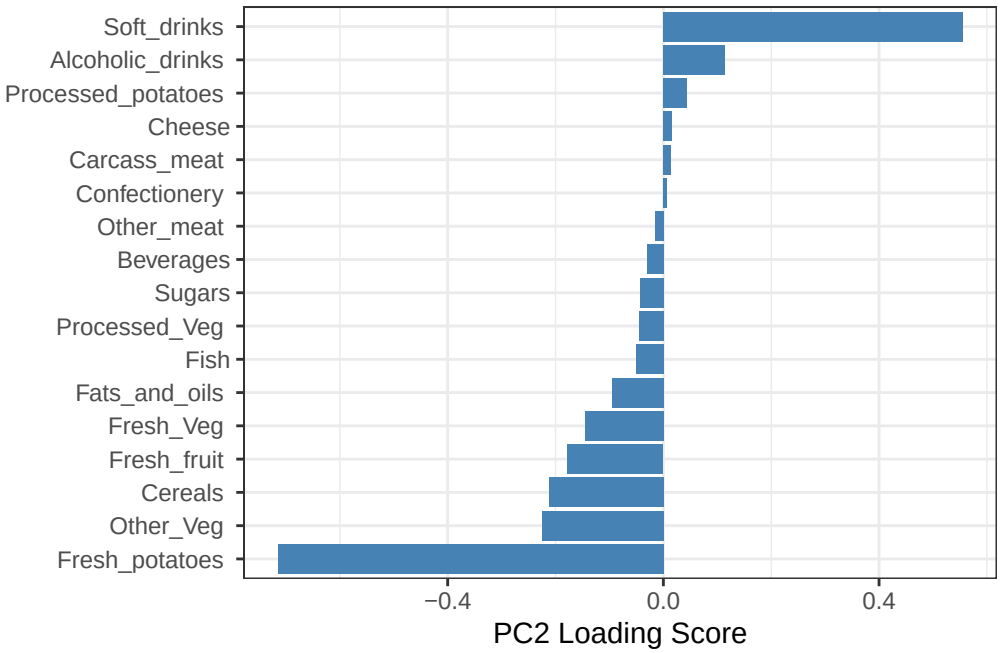
```r
## Lets focus on PC1 as it accounts for > 90% of variance
ggplot(pca$rotation) +
  aes(x = PC1,
      y = reorder(rownames(pca$rotation), PC1)) +
```

```
  geom_col(fill = "steelblue") +
  xlab("PC1 Loading Score") +
  ylab("") +
  theme_bw() +
  theme(axis.text.y = element_text(size = 9))
```



> Q9. Generate a similar 'loadings plot' for PC2. What two food groups feature prominantely and what does PC2 maninly tell us about?

```
ggplot(pca$rotation) +
  aes(x = PC2,
      y = reorder(rownames(pca$rotation), PC2)) +
  geom_col(fill = "steelblue") +
  xlab("PC2 Loading Score") +
  ylab("") +
  theme_bw() +
  theme(axis.text.y = element_text(size = 9))
```



The two food groups "fresh potatoes" and "soft drinks" are the main variables that contribute most and are featured prominently in PC2. PC2 mainly tells us that the observation, soft_drinks, with the largest positive loading score, effectively "pushes" N.Ireland to the right positive side of the plot, while fresh potatoes is the main observation (food) with a high negative score that pushes the other countries to the left side of the plot.

> Key-Point: PCA has the awesome ability to effectively reduce the dimensionality of our data set down from 17 to 2, allowing us to assert (using our figures above) that countries England, Wales and Scotland are 'similar' with Northern Ireland being different in some way. Furthermore, digging deeper into the loadings we were able to associate certain food types with each cluster of countries. This is super useful!